

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

07 - Fetch API e JSON

Prof. Alex Kutzke

Outubro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Escrever dados em JSON, e apontar o que muda em relação a um objeto JavaScript;
2. Converter entre texto JSON e valor com `JSON.parse` e `JSON.stringify`;
3. Servir as páginas por um servidor local, e explicar por que o `fetch` falha em `file://`;
4. Descrever o que `fetch` devolve, e usar `await` dentro de uma função `async`;
5. Conferir `response.ok` e explicar por que um 404 não cai no `catch`;
6. Carregar o catálogo de um arquivo JSON;
7. Consultar uma API pública e tratar a resposta de erro dela.

Roteiro

Requisição

Servidor

JSON

Assíncrono

Fetch

API

Uma requisição pelo console

https://viacep.com.br aberto, F12, aba **Network** aberta, depois aba **Console**:

```
const resposta = await
↪ fetch("https://viacep.com.br/ws/80060000/json/");
resposta.status           // 200
const dados = await resposta.json();
dados.logradouro          // "Rua XV de Novembro"
dados.localidade          // "Curitiba"
```

A página não recarregou.

O que apareceu na aba Network

Uma linha nova, com:

- o endereço pedido;
- o método GET;
- o status 200;
- na aba **Resposta**, um texto entre chaves, com pares de nome e valor.

O JavaScript fez uma requisição HTTP, como as da aula de HTTP, e transformou o corpo da resposta em objeto.

Nesta aula: de onde a página pode fazer requisições, o formato do corpo, por que o `await`, e o `fetch` em si.

Página aberta do disco

Dois cliques no `catalogo.html`: o endereço começa com `file://`. Sem servidor e sem HTTP.

Para HTML, CSS e imagens, basta. Para o `fetch`, o navegador bloqueia:

```
Access to fetch at 'file:///.../produtos.json' from origin 'null'
has been blocked by CORS policy: Cross origin requests are only
supported for protocol schemes: chrome, chrome-extension,
chrome-untrusted, data, http, https, isolated-app.
```

A partir desta aula, as páginas são abertas por um servidor local, em `http://localhost`.

Servidor local

Duas opções:

- VS Code: extensão **Live Server**, botão *Go Live* no canto inferior direito. Recarrega a página a cada alteração salva.
- Terminal, dentro da pasta do repositório:

```
python3 -m http.server 8000
```

e no navegador:

`http://localhost:8000/catalogo.html`

Ctrl+C encerra o servidor.

A aba **Network** passa a mostrar cada arquivo da página como uma requisição, com o status de cada um.

Prática: o ambiente

No seu *fork* de `ds122-lab-02-assignment`:

1. copie suas páginas, `css/`, `imagens/`, `js/produtos.js` e `js/app.js` da tarefa de DOM, ou mova o conteúdo de `base/` para a raiz;
2. inicie o servidor local;
3. abra o catálogo por `http://localhost`;
4. na aba **Network**, confira o status de cada arquivo.

Tempo: 5 minutos.

Resposta: o ambiente

Os cartões aparecem, e a aba **Network** lista:

catalogo.html	200
estilo.css	200
produtos.js	200
app.js	200
produto.svg	200

Um arquivo com 404 é um caminho errado no HTML.

JSON

JavaScript Object Notation: formato de texto para dados, lido e escrito por praticamente qualquer linguagem.

```
[
  {
    "id": 1,
    "nome": "Café em grão",
    "preco": 32.5,
    "disponivel": true,
    "tags": ["orgânico", "torra média"]
  }
]
```

Um único valor: objeto, array, string, número, true, false ou null, um dentro do outro.

O que muda em relação a um objeto JavaScript

No JavaScript	No JSON
nome do campo com ou sem aspas	sempre entre aspas duplas
aspas simples, duplas ou crase	só aspas duplas
vírgula depois do último item	erro
comentários	não há
<code>const produtos = antes</code>	só o valor
funções, <code>undefined</code>	não existem

"preco": "32.5" é uma string. "preco": 32.5 é um número, e só ele soma.

JSON.parse e JSON.stringify

O JSON chega pela rede como texto.

```
const texto = '{"nome": "Café em grão", "preco": 32.5}';
```

```
const produto = JSON.parse(texto);  
produto.preco * 2;      // 65
```

```
JSON.stringify({ nome: "Mel", preco: 24.9 });  
// '{"nome": "Mel", "preco": 24.9}'
```

Texto com erro de sintaxe lança `SyntaxError`. Teste:

```
JSON.parse("{'nome': 'Mel'}");
```

Na prática, o `response.json()` faz essa conversão por nós.

Prática: os produtos em JSON

1. crie o `produtos.json` na raiz, com os produtos do `js/produtos.js`;
2. acrescente a cada produto o campo `descricao`;
3. abra `http://localhost:8000/produtos.json` no Firefox e confira que ele mostra os dados formatados;
4. apague o `js/produtos.js` e a tag `<script>` dele.

Confira que a grade ficou vazia, com um erro no console.

Tempo: 10 minutos.

Resposta: os produtos em JSON

```
[
  {
    "id": 1,
    "nome": "Café em grão",
    "descricao": "Grãos de torra média, de produção familiar.",
    "categoria": "Bebidas",
    "preco": 32.5,
    "desconto": 0.15,
    "imagem": "imagens/produto.svg"
  }
]
```

ReferenceError: produtos is not defined

O `renderiza(produtos)` do fim do `app.js` ainda procura o array.

Operações assíncronas

Uma requisição leva tempo: milissegundos no servidor local, segundos numa conexão ruim.

Se o JavaScript parasse até a resposta chegar, a página congelaria: sem rolar, sem responder a cliques.

Por isso o `fetch` dispara a requisição e devolve, na hora, uma *Promise* (promessa): um objeto que representa um resultado que ainda vai chegar.

Promise

```
const p = fetch("https://viacep.com.br/ws/80060000/json/");  
p    // Promise {<pending>}
```

Três estados:

- pendente (*pending*): a resposta ainda não chegou;
- cumprida (*fulfilled*): a resposta está dentro;
- rejeitada (*rejected*): a requisição falhou, e o erro está dentro.

await

`await` antes de uma *Promise* espera que ela se resolva e devolve o valor que está dentro:

```
const resposta = await  
  ↪ fetch("https://viacep.com.br/ws/80060000/json/");  
resposta    // Response {status: 200, ok: true, ...}
```

Fora do console, `await` só pode ser usado dentro de uma função marcada com `async`.

O objeto Response

Propriedade ou método	O que é
<code>resposta.status</code>	código de status HTTP, como 200 ou 404
<code>resposta.ok</code>	true com status entre 200 e 299
<code>resposta.json()</code>	lê o corpo e converte de JSON; devolve uma <i>Promise</i>
<code>resposta.text()</code>	lê o corpo como texto; devolve uma <i>Promise</i>

O endereço pode ser completo, como o do ViaCEP, ou relativo à página, como "produtos.json".

response.ok e o 404

O fetch só rejeita quando a requisição não acontece: sem rede, servidor fora do ar, domínio que não existe, bloqueio do navegador.

Um 404 é uma resposta HTTP completa:

```
const r = await fetch("nao-existe.json");  
r.status    // 404  
r.ok        // false
```

Nenhum erro vermelho. Todo fetch confere `response.ok` antes de usar o corpo.

response.json()

Ler o corpo também é assíncrono: `json()` devolve outra *Promise*.

```
const resposta = await fetch("produtos.json");  
const produtos = await resposta.json();  
produtos.length // 6
```

Sem o segundo `await`, `produtos` é uma *Promise*:

```
TypeError: produtos.forEach is not a function
```

Uma função async com try e catch

```
const carregaProdutos = async () => {
  try {
    const resposta = await fetch("produtos.json");
    if (!resposta.ok) {
      throw new Error(`Erro HTTP: ${resposta.status}`);
    }
    const produtos = await resposta.json();
    renderiza(produtos);
  } catch (erro) {
    console.error(erro);
    const aviso = document.createElement("p");
    aviso.classList.add("erro");
    aviso.textContent = "Não foi possível carregar os produtos.";
    grade.replaceChildren(aviso);
  }
};

carregaProdutos();
```

Linha a linha

- `async` marca a função como assíncrona e permite `await` dentro dela;
- o `try` contém o caminho normal; qualquer erro desvia para o `catch`;
- `throw new Error(...)` lança um erro de propósito, e o 404 cai no mesmo `catch` que a falha de rede;
- o `catch` registra no console e avisa na página, porque o visitante não abre o console;
- `renderiza` é a da aula de DOM, e não sabe de onde a lista veio.

Prática: o catálogo pelo fetch

1. no fim do `app.js`, troque `renderiza(produtos);` por uma `carregaProdutos` com `fetch`, `await` e `response.ok`, ainda sem `try/catch`;
2. na `criaCartao`, acrescente um `<p class="descricao">` logo depois da imagem;
3. acrescente um sétimo produto ao `produtos.json`.

Confira que os cartões voltam com a descrição, que o sétimo aparece sem mexer no JavaScript, e que a aba **Network** mostra o `produtos.json` com 200.

Tempo: 15 minutos.

Resposta: o catálogo pelo fetch

```
const carregaProdutos = async () => {  
  const resposta = await fetch("produtos.json");  
  if (!resposta.ok) {  
    throw new Error(`Erro HTTP: ${resposta.status}`);  
  }  
  const produtos = await resposta.json();  
  renderiza(produtos);  
};
```

```
carregaProdutos();
```

```
// na criaCartao, depois da imagem  
const descricao = document.createElement("p");  
descricao.classList.add("descricao");  
descricao.textContent = produto.descricao;  
cartao.append(descricao);
```

Prática: a mensagem de erro

1. envolva o corpo da `carregaProdutos` em `try/catch`;
2. no `catch`, mostre Não foi possível carregar os produtos. em um `<p class="erro">` dentro da grade;
3. troque `"produtos.json"` por `"produto.json"` no `fetch` e recarregue.

Confira a mensagem na página, o erro no console e o 404 na aba **Network**. Desfaça a troca.

Tempo: 10 minutos.

Resposta: a mensagem de erro

O código do slide *Uma função async com try e catch*.

Com "produto.json":

Network: produto.json 404

Console: Error: Erro HTTP: 404

Página: Não foi possível carregar os produtos.

Sem o `if (!resposta.ok)`, o `json()` tenta ler a página de erro do servidor como JSON, e o `catch` recebe um `SyntaxError` que não diz que o arquivo não foi encontrado.

A ordem de execução

```
let produtos = [];  
  
const carrega = async () => {  
  const resposta = await fetch("produtos.json");  
  produtos = await resposta.json();  
  console.log("2. dados chegaram:", produtos.length);  
};  
  
carrega();  
console.log("1. logo depois da chamada:", produtos.length);
```

-
1. logo depois da chamada: 0
 2. dados chegaram: 6
-

No primeiro `await`, a função devolve o controle. O que usa os dados vai **dentro** dela, depois do `await`.

Uma API pública: ViaCEP

API, na web: um endereço que responde dados em vez de páginas, em geral em JSON.

`https://viacep.com.br/ws/80060000/json/`

Pedido	Status	Corpo
123, formato inválido	400	página de erro em HTML
99999999, não existe	200	{ "erro": "true" }

No segundo caso, `response.ok` é `true`. O erro está no conteúdo, e a documentação da API diz como.

Formato do CEP e CORS

O formato pode ser conferido antes da requisição:

```
const cep = campoCep.value.replace("-", "");  
if (cep.length !== 8 || Number.isNaN(Number(cep))) {  
  // mostra a mensagem e não faz o fetch  
}
```

O navegador só entrega a resposta de outro site se o servidor permitir, pelo cabeçalho `Access-Control-Allow-Origin`. É o CORS (*Cross-Origin Resource Sharing*), o mesmo mecanismo que bloqueia `file://`. O ViaCEP permite qualquer site.

Prática: o endereço pelo CEP

1. no `contato.html`, acrescente os campos `cep`, `rua`, `bairro` e `cidade`, e um `<p id="erro-cep" class="erro">`;
2. crie o `js/contato.js`, ligado com `defer`;
3. escreva a `buscaCep`, chamada no `blur` do `cep`, que confere o formato, faz o `fetch` e preenche os três campos.

Confira com 80060000, 123 e 99999999.

Tempo: 10 minutos.

Resposta: o endereço pelo CEP

```
const campoCep = document.querySelector("#cep");
const erroCep = document.querySelector("#erro-cep");
const buscaCep = async () => {
  erroCep.textContent = "";
  const cep = campoCep.value.replace("-", "");
  if (cep.length !== 8 || Number.isNaN(Number(cep))) {
    erroCep.textContent = "Digite um CEP com oito números.";
    return;
  }
  const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);
  const dados = await resposta.json();
  document.querySelector("#rua").value = dados.logradouro;
  document.querySelector("#bairro").value = dados.bairro;
  document.querySelector("#cidade").value = dados.localidade;
};
campoCep.addEventListener("blur", buscaCep);
```

Com 999999999, os campos ficam com undefined. O tratamento do erro e o try/catch ficam para a tarefa.

Quando dá errado

Sintoma	Causa provável
<code>blocked by CORS policy, com file://</code>	página sem servidor local
<code>SyntaxError</code> ao ler o JSON	vírgula sobrando, aspas simples, comentário
<code>.forEach is not a function</code>	<code>await</code> esquecido no <code>json()</code>
<code>await is only valid in async functions</code>	<code>await</code> fora de função <code>async</code>
<code>Unexpected token '<', com 404</code>	<code>json()</code> lendo a página de erro, sem conferir <code>response.ok</code>
total ou contagem zerados	dados usados fora da função
soma dá "032.518"	números entre aspas no JSON

Os dois últimos não aparecem no console. A aba **Network** mostra o status e o corpo que chegou.

Tarefa

Os exercícios valem nota e compõem o item *Exercícios em sala*.

Enunciado no README.md de ds122-lab-02-assignment; prazo na UFPR Virtual. As paradas desta aula cobrem os exercícios 1 a 4; o exercício 5 é de revisão para a Prova 2.

Entrega pelo *fork* no GitLab, individual ou em dupla, sem uso de IA generativa para produzir o código.

O catálogo pelo produtos.json é o requisito 4 da Entrega 2, com prazo no domingo, 11/10. No trabalho, o cartão mantém o que a Entrega 1 pedia: imagem, nome, descrição curta e preço.